OPERATING SYSTEM

Unit – I

Prepared by:

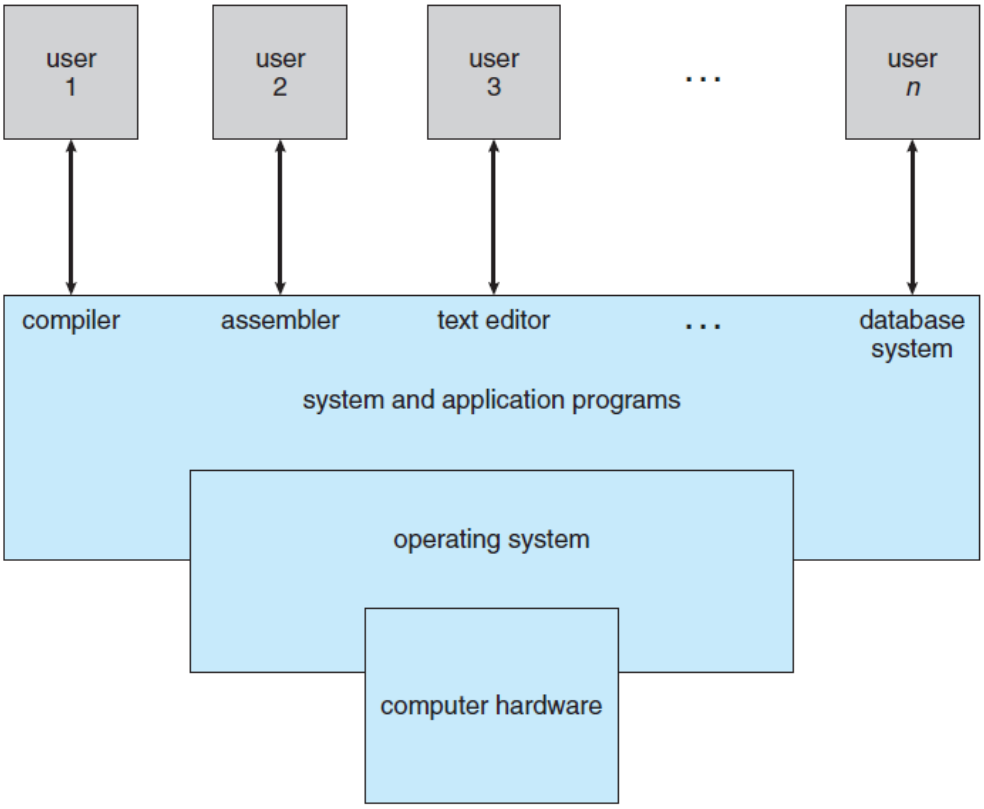
by- E-Learnify

Introduction to Operating System

An **operating system** is a program that manages a computer's hardware.

It also provides a basis for application programs and acts as an **intermediary** between the **computer user** and the **computer hardware**.

Some operating systems are designed to be **convenient,** others to be **efficient,** and others to be some **combination of the two**.



[Abstract view of the components of a computer system]

1. What Operating Systems Do

A computer system can be divided roughly into four components: the **hardware,** the **operating system,** the **application programs** and the **users.**
The operating system controls the hardware and coordinates its use among the various application programs for the various users.
An operating system is similar to **government**; performs no useful function by itself. It simply provides an **environment** within which other programs can do useful work.

User View of Operating Systems

Most computer users sit in front of a **PC** (Personal Computer), consisting of a monitor, keyboard, mouse and system unit. Such a system is designed for one user to **monopolize** its resources.
In this case, the operating system is designed mostly for **ease of use**, with some attention paid to **performance** and none paid to **resource utilization.**
In other cases, a user sits at a terminal connected to a **mainframe** or a **minicomputer**.
These users share resources and may exchange information. The operating system in such cases is designed to **maximize resource utilization**.
The user interface for **mobile computers** generally features a **touch screen**, where the user interacts with the system by pressing and swiping fingers across the screen.
Operating for **embedded computers** in home devices and automobiles designed to run **without user intervention**.

System View of Operating Systems

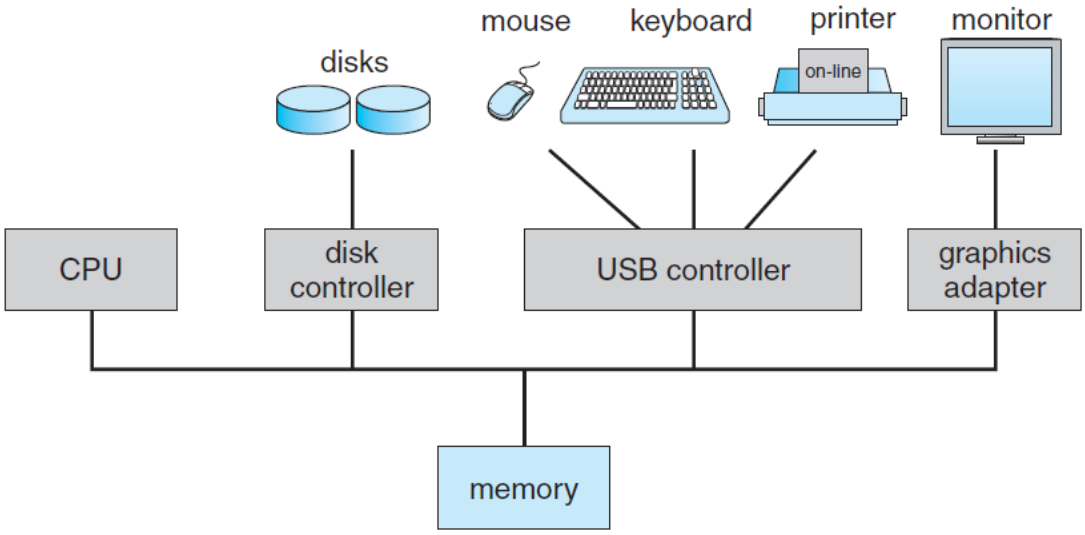
From the computer's point of view, we can see the operating system as a **resource allocator.**
A computer system has many resources that may be required to solve a problem: CPU time, memory space, file-storage space, I/O devices and so on.
The operating system acts as the **manager** of these resources.
An operating system is a **control program**. A control program manages the execution of user programs to prevent errors and improper use of the computer.
It is especially concernedwith the **operation** and **control** of I/O devices.

Definition of Operating Systems

A common definition of operating system is that: "*the operating system is the one program running at all times on the computer - usually called* ***the kernel***".

2. Computer-System Organization

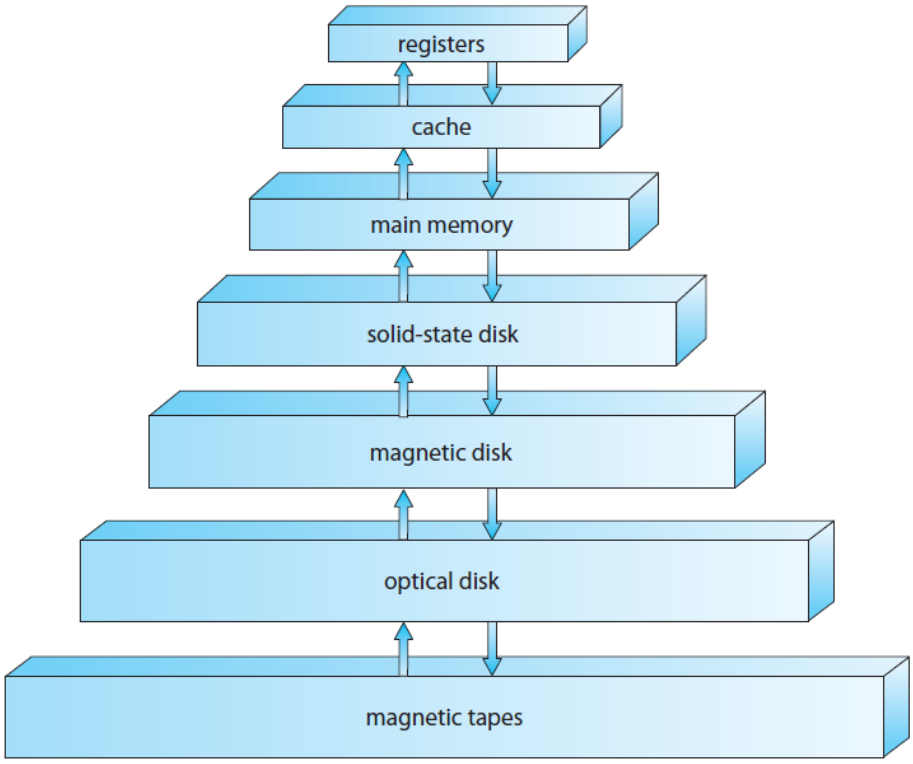
A modern general-purpose computer system consists of one or more CPUs and a number of device controllers connected through a **common bus** that provides access to shared memory.
For a computer to start running, for instance when it is powered up, it needs to have an initial program to run called **bootstrap program**.
It is stored within the computer hardware in read-only memory (**ROM**) and loads OS from secondary storage device to main memory (**RAM**).
When the system is fully booted the system waits for some **events to occur**.
The occurrence of an event is usually signalled by an **interrupt** from either the hardware or the software.
Hardware may trigger an interrupt at any time by sending a signal to the CPU, usually by way of the system bus.
Software may trigger an interrupt by executing a special operation called a **system call** (also called a **monitor call**).
When the CPU is interrupted, it stops what it is doing and immediately transfers execution to a **fixed location**.
The fixed location usually contains the **starting address** where the service routine for the interrupt is located.
The interrupt service routine executes the service routine and on completion, the CPU resumes the interrupted computation.



[A modern computer system]

Storage Structure

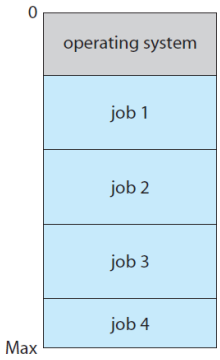
The CPU can load instructions only from memory, so any programs to run must be stored there. This memory is called main memory (also called random-access memory or **RAM**).
Ideally, we want the programs and data to reside in main memory permanently. This arrangement usually is not possible for the following two reasons:
    - Main memory is usually too small to store all needed programs and data.
    - Main memory is a **volatile** that is it loses its contents when power is off.
Thus, most computer systems provide **secondary storage** as an extension of main memory to store large quantities of data permanently.
The most common secondary-storage device is a **magnetic disk**, which provides storage for both programs and data.
Other memory technologies include - **cache memory**, **CD-ROM**, **magnetic tapes** etc.



[Storage-device hierarchy]

3. Operating-System Structure

An operating system provides the **environment** within which programs are executed.
A single program cannot keep either the CPU or the I/O devices busy at all times.
This requires **multiprogramming**. Multiprogramming increases CPU utilization by organizing jobs so that the CPU always has one job to execute.
Multiprogramming requires several jobs in main memory simultaneously. Since main memory is too small to accommodate all jobs, **the jobs are kept initially on the disk**.



[Memory layout for a multiprogramming system]

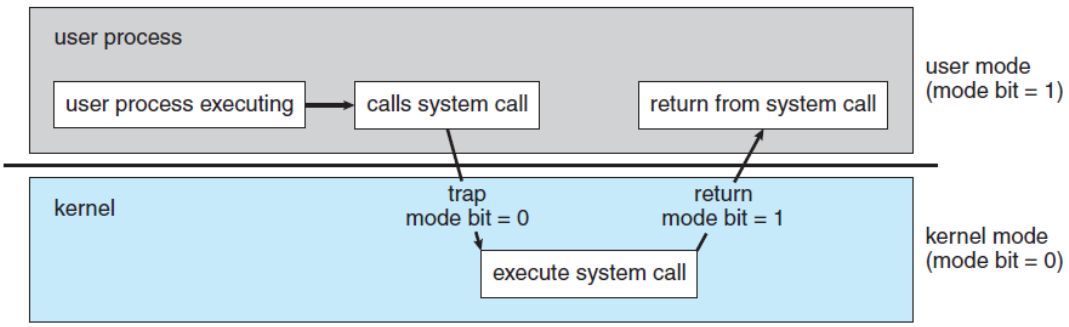
The operating system picks and begins to execute one of the jobs in memory. The job may have to wait for some task, such as an I/O operation, to complete.
In a non-multiprogrammed system, the CPU would sit **idle**. In a multiprogrammed system, the operating system simply switches to, and executes, another job. When that job needs to wait, the CPU switches to another job, and so on.
Time sharing (or **multitasking**) is a logical extension of multiprogramming. In time-sharing systems, the CPU executes multiple jobs by switching among them, but the switches occur so frequently that the users can interact with each program while it is running.
A time-shared operating system uses **CPU scheduling** and multiprogramming to provide each user with a small portion of a time-shared computer.
A program loaded into memory and executing is called a **process**.
Time sharing and multiprogramming require that several jobs be kept simultaneously in memory. If several jobs are ready to be brought into memory, and if there is not enough room for all of them, then the system must choose among them. This decision is called **job scheduling.**
Having several programs in memory at the same time requires some form of **memory management**.
If several jobs are ready to run at the same time, the system must choose which job will run first. This decision is called **CPU scheduling.**

4. Operating-System Operations

Modern operating systems are **interrupt** driven. Events are almost always signalled by the occurrence of an interrupt or a **trap**. A trap (or an exception) is a software-generated interrupt causedby an error.
For each type of interrupt, separate segments of code in the operating system determine what action should be taken. An **interrupt service routine** is provided to deal with the interrupt.
Since the operating system and the users **share** the hardware and software resources of the computer system, we need to make sure that an error in a user program should not affect other running program.
A properly designed operating system must ensure that an incorrect (or malicious) program cannot cause other programs to execute incorrectly.

Dual-Mode of Operation

In order to ensure the proper execution of the operating system, we must be able to distinguish between the execution of **operating-system code** and **user defined code**.
So we need two separate modes of operation: **user mode** and **kernel mode** (also called **supervisor mode**, **system mode**, or **privileged mode**).



[Transition from user to kernel mode]

A bit, called the **mode bit**, is added to the hardware of the computer to indicate the current mode: **kernel (0)** or **user (1).**
The dual mode of operation provides us with the means for protecting the operating system from errant users and errant users from one another.
This protection can be implemented by designating some of the machine instructions that may cause harm as **privileged instructions**.
The hardware allows privileged instructions to be executed only in **kernel mode**.

If an attempt is made to execute a privileged instruction in user mode, the hardware does not execute the instruction but rather treats it as illegal and traps it to the operating system.
The instruction **to switch to kernel mode** is an example of **a privileged instruction**. Some other examples include I/O control, timer management, and interrupt management.

5. Process Management

A program does nothing unless its instructions are executed by a CPU. So a program in execution is a **process**.
A process needs certain resources - including CPU time, memory, files,and I/O devices - to accomplish its task.
These resources are either given to the process when it is created or allocated to it while it is running.
Process is the **unit of work** in a system.
A system consists of a collection of processes, some of which are **operating-system processes** (those that execute operating system code) and the rest of which are **user processes.**
The operating system is responsible for the following activities in connection with process management:

    - Scheduling processes and threads on the CPUs
    - Creating and deleting both user and system processes
    - Suspending and resuming processes
    - Providing mechanisms for process synchronization
    - Providing mechanisms for process communication

6. Memory Management

Main memory is a repository of quickly accessible data shared by the CPU and I/O devices.
The CPU reads instructions from main memory during the **instruction-fetch cycle** and both reads and writes data from main memory during the **data-fetch cycle.**
To improve both the utilization of the CPU and the speed of the computer's response to its users, general-purpose computers must keep several programs in memory, creating a need for memory management.

The operating system is responsible for the following activities in connection with memory management:

- Keeping track of which parts of memory are currently being used and who is using them
- Deciding which processes (or parts of processes) and data to move into and out of memory
- Allocating and de-allocating memory space as needed

7. Storage Management

File-System Management

A file is a **collection of related information** defined by its creator. Data files maybe numeric, alphabetic, alphanumeric or binary.
The operating system implements the abstract concept of a file by managing mass-storage media, such as tapes and disks and the devices that control them.
When **multiple users** have access to files, it may be desirable to control **which user** may access a file and **how that user** may access it (for example, read, write, append).
The operating system is responsible for the following activities in connectionwith file management:

- Creating and deleting files
- Creating and deleting directories to organize files
- Supporting primitives for manipulating files and directories
- Mapping files onto secondary storage
- Backing up files on stable (non-volatile) storage media

Mass-Storage Management

Because the main memory is too small to accommodateall data and programs, the computer system must provide secondary storage **to back up main memory**.
Most modern computer systems use disks as the principal on-line storage medium for both programs and data.
The operating system is responsible for the following activities in connection with disk management:

- Free-space management
- Storage allocation
- Disk scheduling

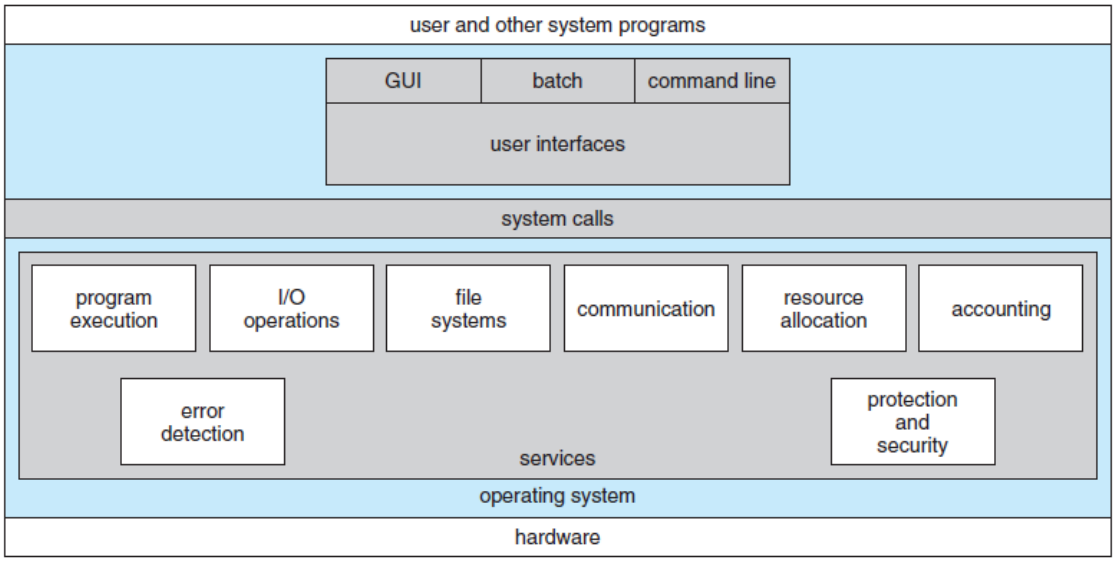
8. Protection and Security

If a computer system has multiple users and allows the concurrent execution of multiple processes, then access to data must be regulated.
For that purpose **mechanisms ensure** that files, memory segments, CPU, and other resources can be operated on by only those processes that have gained **proper authorization** from the operating system.
Protection is any mechanism for controlling the access of processesor users to the resources defined by a computer system.
A system can have adequate protection but still be prone to failure and allow inappropriate access.
It is the job of **security** to defend a system from **external and internal attacks**.
Protection and security require the system to be able to distinguish among all its users.
Most operating systems maintain a list of user names andassociated user identifiers (user IDs).
In some circumstances, we wish to distinguish among sets of users rather than individual users.
To accomplish this, we need to define a group name and the set of users belonging to that group. Group functionality canbe implemented as a system-wide list of group names and group identifiers.

System Structures

1. Operating system services

An operating system provides an environment for the **execution of programs**.
It provides certain services to programs and to the users of those programs.



[A view of operating system services]

The following are the services provided by operating system to the users:

User interface:

Almost all operating systems have a user interface (UI). This interface can take several forms.
One form is **Command-line interface (CLI)** which uses text commands and it provides a way for entering them. For example - **Command prompt** in Windows and **Terminal** for Linux environment.
Another form is **Batch interface** that receives files which contains commands and executes one by one.
One more form is **Graphical user interface (GUI).** This is a window system with a pointing device to direct I/O, choose from menus and make selections and a keyboard to enter text.

Program execution:

The system must be able to load a program into memory and to run that program. The program must be able to end its execution, either normally or abnormally (indicating error).

I/O operations:

A running program may require I/O resources like file or an I/O device.
For specific devices, special functions may be desired (such as recording to a CD or DVD drive).
Therefore, the operating system must provide away to do I/O operation to the running program.

File-system manipulation:

Operating system provides functionalities for create, open, read, write, close and delete files.

Communications:

There are many circumstances in which one processneeds to exchange information with another process.
Such communication may occur between processes that are executing on the same computer or between processes that are executing on different computer systems tied together by a computer network.
Communications may be implemented via **shared memory**, in which two or more processes read and write to a shared section of memory, or **message passing**, in which packets of information in predefined formats are moved between processes by the operating system.

Error detection:

Errors may occur in the CPU and memory hardware, in I/O devices and in the user program (such as an arithmetic overflow, an attempt to access an illegal memory location, or a too-great use of CPU time).
For each type of error, the operating system should take the appropriate action to ensure **correct and consistent computing.**
Sometimes, it has no choice but to **halt the system**. At other times, it might terminate an error-causing process or return an error code to a process for the process to detect and possibly correct.

Resource allocation:

When there are multiple users or multiple jobs running at the same time, resources must be allocated to each of them.
The operating system manages many different types of resources such as CPU cycles, main memory, and file storage.

Accounting:

Operating system keeps track of **which users** use **how much** and **what kinds** of computer resources.

Protection and security:

Operating system provides various protection and security mechanism to ensure that only authorized process can access resource for execution.

2. User and Operating-System Interface

There are two fundamental ways for users to **interface with the operating system**.
One is **Command-Line Interface**,(CLI) or command interpreter, that allows users to directly enter commands to be performed by the operating system.
The other allows users to interface with the operating system via a **Graphical User Interface** (GUI).

Command Interpreters

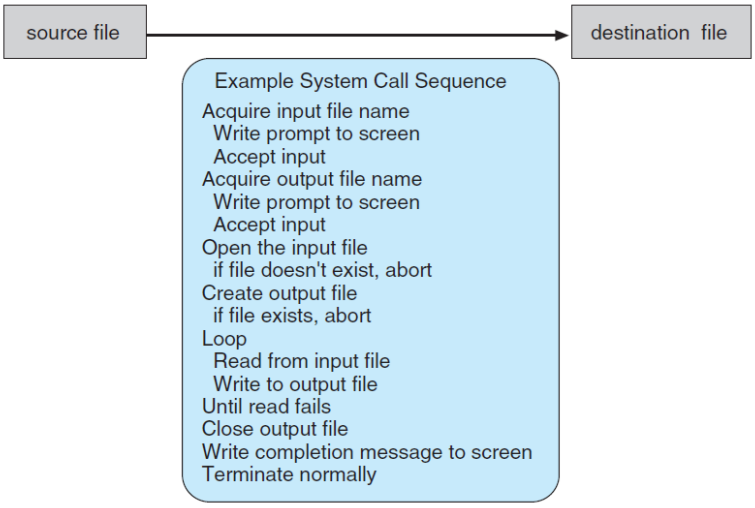
On UNIX and Linux systems, there are several different command interpreters are present and these are called **shells**. These include the Bourne shell, C-shell, Bourne-Again shell, Korn shell and others.
The main function of the command interpreter is to get and execute the next user-specified command. Many of the commands given at this level manipulate files: create, delete, list, print, copy, execute, and so on.
These commands can be implemented in two general ways. In one approach, the command interpreter itself contains the code to execute the command.
An alternative approach - used by UNIX, among other operating systems - implements most commands through system programs.
The UNIX command to delete a file
    - rm file.txt
This would search for a file called rm, load the file into memory, and execute it with the parameter file.txt.

Graphical User Interfaces

In GUI users employ a mouse-based window-and-menu system characterized by a desktop metaphor.
The user moves the mouse to position its pointer on images and **icons** on the screen (the desktop) that represent programs, files, directories and system functions.
Depending on the mouse pointer's location, clicking a button on the mouse can invoke a program, select a file or directory - known as a **folder** - or pull down a menu that contains commands.
Graphical user interfaces first appeared due in part to research taking place in the early 1970s at **Xerox PARC** research facility. The first GUI appeared on the **Xerox Alto** computer in 1973.
However, graphical interfaces became more widespread with the advent of Apple Macintosh computers in the 1980s.
UNIX systems have been dominated by command-line interfaces. Most Windows users are happy to use the Windows GUI environment and almost never use the MS-DOS shell interface.
The user interface can vary from system to system even from user to user within a system. The design of a useful and user-friendly user interface is therefore not a direct function of the operating system.

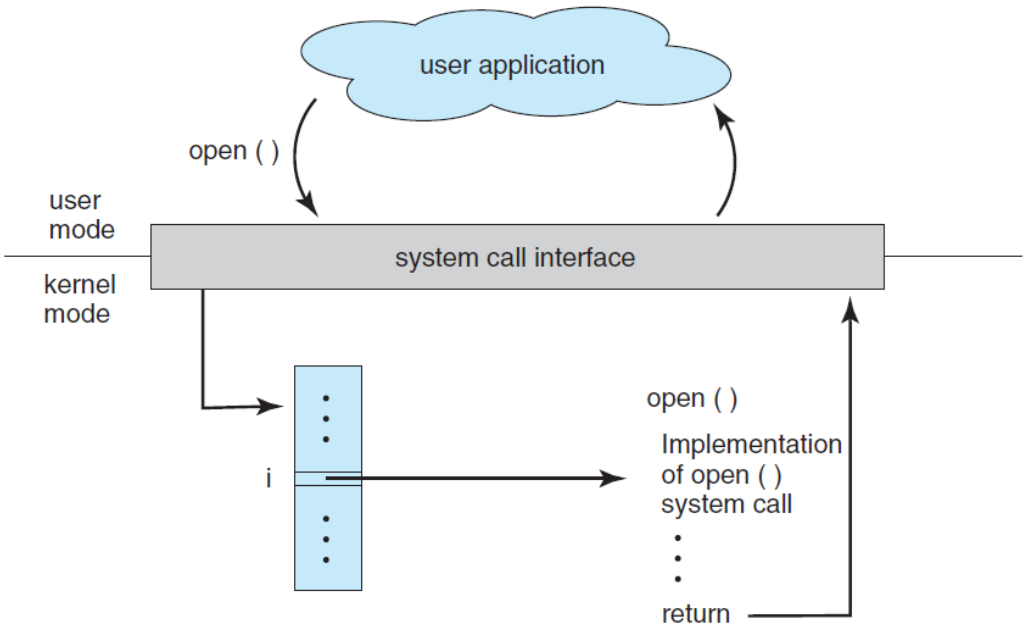
3. System Calls

System calls provide an **interface to the services** provided by an OS. These calls are generally available as **routines** written in C, C++ and some may be using assembly-language instructions.
Let's first use an example to illustrate how system calls are used: A simple program to read data from one file and copy them to another file.



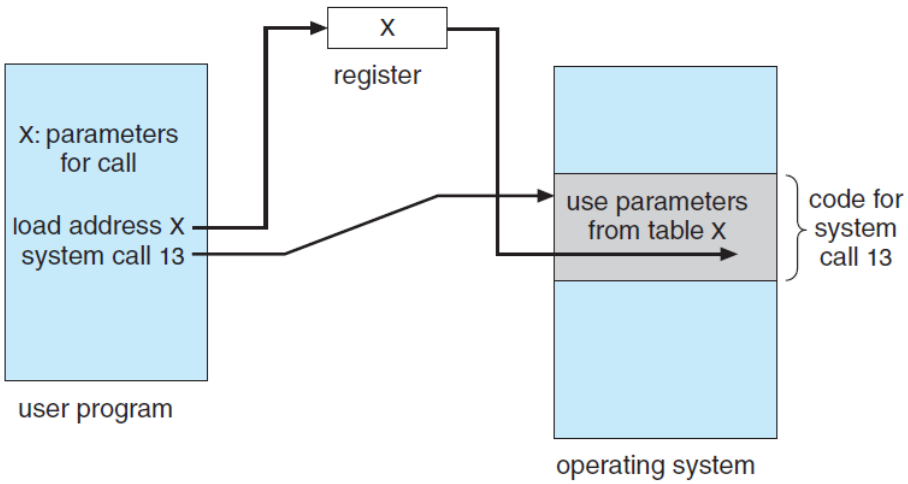
[Example of how system calls are used]

Operating systems execute thousands of system calls per second.
System calls are made available to application programmers as an **application programming interface** (API).
The API specifies a set of functions that are available to an application programmer, including the parameters that are passed to each function and the return values the programmer can expect.
There are three most common APIs available to application programmers; **Windows API** for Windows systems, the **POSIX API** for UNIX and Linux systems and the **Java API** for programs that run on the Java virtual machine.
A programmer accesses an API via a library of code provided by the operating system.
Behind the scenes, the functions that make up an API typically invoke the actual system calls on behalf of the application programmer.
For example, the Windows function **CreateProcess()** is used to create a new process actually invokes the **NTCreateProcess()** system call in the Windows kernel.
When a system calls are made the system-call interface intercepts the calls in the API and invokes the necessary system calls within the operating system.
Typically **a number** is associated with each system call, and the system-call interface maintains a table indexed according to these numbers.
The caller need know nothing about how the system call is implemented or what it does during execution.



[The handling of a user application invoking the open() system call]

System calls requires **parameters** to be passed for execution. Three general methods are used to pass parameters to the operating system.
The first approach is to pass the parameters in **registers**. Other approaches are **block** and **stack** method. In these methods there is no limit on the number or length of parameters being passed.
In block method, the parameters are generally stored in a block or **table** in memory and the **address** of the block is passed as a parameter in a register. This is the approach taken by Linux and Solaris.
Parameters also can be placed or pushed onto the stack by the program and popped off the stack by the operating system.



[Passing of parameters as a table]

4. Types of System Calls

System calls can be grouped roughly into five major categories: process control, file manipulation, device manipulation, information maintenance and communications.

Process control:

- end, abort
- load, execute
- create process, terminate process
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

File management:

- create file, delete file
- open, close
- read, write, reposition
- get file attributes, set file attributes

Device management:

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

Information maintenance:

- get time or date, set time or date
- get system data, set system data
- get process, file, or device attributes
- set process, file, or device attributes

Communications:

- create, delete communication connection
- send, receive messages
- transfer status information
- attach or detach remote devices

5. System Programs

Another aspect of a modern system is its collection of **system programs**. System programs, also known as **system utilities**, provide a convenient environment for **program development and execution.**
System programs provide basic functionalities to users so that they do not need to write their own environment for program development (editors, compilers) and program execution (shells).
They can be divided into these categories:

File management:

These programs create, delete, copy, rename, print, dump, list, and generally manipulate files and directories.

Status information:

Some programs simply ask the system for the date, time, amount of available memory or disk space, number of users, or similar status information.
Others are more complex, providing detailed performance, logging and debugging information.
Typically, these programs format and print the output to the terminal or other output devices or files or display it in a window of the GUI.

File modification:

Several text editors may be available to create and modify the content of files stored on disk or other storage devices. There may also be special commands to search contents of files or perform transformations of the text.

Programming-language support:

Compilers, assemblers, debuggers, and interpreters for common programming languages (such as C, C++, Java,and PERL) are often provided with the operating system or available as a separate download.

Program loading and execution:

Once a program is assembled or compiled, it must be loaded into memory to be executed. The system may provide absolute loaders, relocatable loaders, linkage editors and overlay loaders.
Debugging systems for either higher-level languages or machinelanguage are needed as well.

Communications:

These programs provide the mechanism for creating virtual connections among processes, users, and computer systems.
They allow users to send messages to one another's screens, to browse Webpages, to send e-mail messages, to log in remotely, or to transfer files fromone machine to another.

Background services:

All general-purpose systems have methods for launching certain system-program processes at boot time.
Some of these processes terminate after completing their tasks, while others continue to run until the system is halted.
Constantly running system-program processes are known as services, subsystems, or daemons.
Some examples include process schedulers that start processes according to a specified schedule, system error monitoring services and print services.

6. Operating-System Design and Implementation

In this section, we discuss problems we face in designing and implementing an operating system.

Design Goals:

The first problem in designing a system is to define **goals** and **specifications**.
At the highest level the design of the system will be affected by the **choice of hardware** and the **type of system**: i.e. batch, time sharing, single user, multiuser, distributed, real time or general purpose.
The requirements can be divided into two basic groups: **user goals** and **system goals**.
Users want certain obvious properties in a system. The system should be convenient to use, easy to learn and to use, reliable, safe and fast.
The system should be easy to design, implement, and maintain; and it should be flexible, reliable, error free and efficient.

Mechanisms and Policies:

Mechanisms determine **how to do something**; policies determine **what will be done**.
For example, **the timer construct** is a mechanism for ensuring CPU protection, but deciding **how long** the timer is to be set for a particular user is a policy decision.
Policy decisions are important for all resource allocation. Whenever it is necessary to decide whether or not to allocate a resource, a policy decision must be made.
Whenever the question is **how rather than what**, it is a **mechanism** that must be determined.

Implementation:

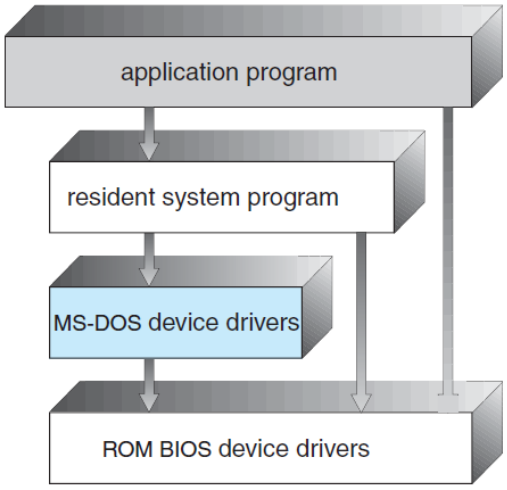
Once an operating system is designed, it must be implemented.
Because operating systems are **collections of many programs**, written by many people over a long period of time, it is difficult to make general statements about how they are implemented.
Early operating systems were written in **assembly language**.
Now, although some operating systems are still written in assembly language, most are written in a higher-level language such as **C** or an even higher-level language such as **C++.**

7. Operating-System Structure

A modern operating system is a large complex system so it must be engineered carefully to function properly and be modified easily.
The following are commonly used approaches to design operating systems.

Simple Structure:

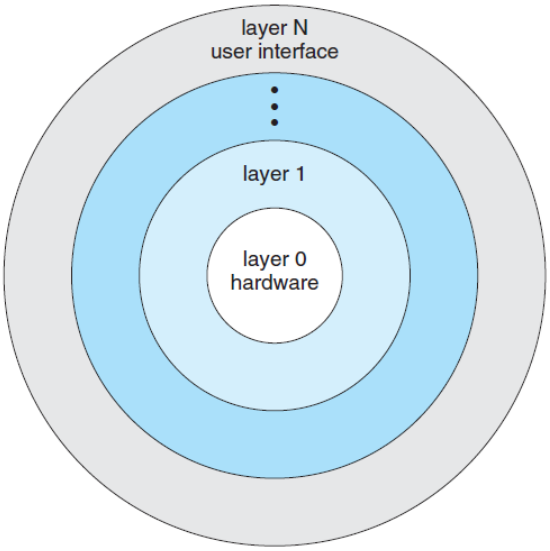
Many operating systems do not have well-defined structures. Such systems started as small, simple, and limited systems and then grew beyond their original scope.
MS-DOS operating system was developed using simple structure.
It was originally designed and implemented by few people who had no idea that it would be so popular.
It was written to provide the most functionality in the least space, so it was not divided carefully.



[MS-DOS layer structure]

Layered Approach:

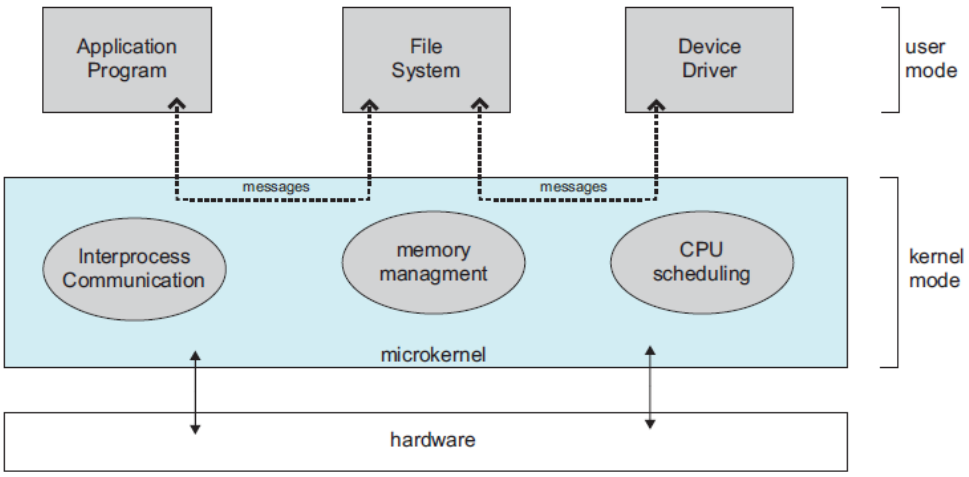
In layered approach the operating system is broken into a number of layers (levels).
The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface.
A typical operating-system layer (say layer M), consists of data structures and a set of routines that can be invoked by higher-level layers.
Layer M, in turn, can invoke operations on lower-level layers.
The main advantage of the layered approach is **simplicity of construction and debugging.**
The major difficulty with the layered approach involves **appropriately defining the various layers.**



[A layered operating system]

Microkernels:

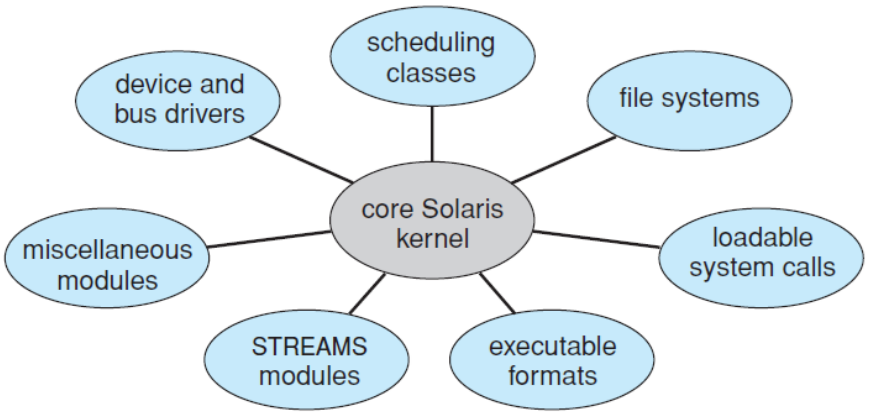
This method structures the operating system by removing all nonessential components from the kernel and implementing them as system and user-level programs.
The result is a **smaller kernel**.
Microkernels provide minimal process and memory management, in addition to a communication facility.
One benefit of the microkernel approach is that it makes **extending the operating system easier.**
Unfortunately, the **performance** of microkernels can suffer due to increased system-function overhead.



[Architecture of a typical microkernel]

Modules:

The best current methodology for operating-system design involves using **loadable** kernel modules.
Here, the kernel has a set of core components and links in additional services via modules, either at boot time or during run time.
This type of design is common in modern implementations of UNIX, such as Solaris, Linux, and Mac OS X, as well as Windows.
The idea of the design is for the kernel to provide core services while other services are implemented dynamically, as the kernel is running.



[Solaris loadable modules]

Adopted from the book:
Operating System Concepts
By:
Abraham Silberschatz,
Peter B. Galvin,
Greg Gagne,
Eighth Edition, Wiley Student Edition 2009.